

Gradient Boosting Regressor — Laptop Price Prediction

Python 3.10 · scikit-learn (GradientBoostingRegressor) · Author: Sambit7848

A hands-on regression project that predicts laptop prices from hardware specs using **Gradient Boosting Regressor**, and compares its performance against a baseline **Linear Regression** model to show the value of a non-linear, ensemble approach.

Dataset: `laptop_price_gbr.csv` · **Shape:** (10000, 6) · **Inputs:** `RAM_GB`, `Storage_GB`, `ScreenSize_inch`, `Weight_kg`, `BatteryLife_hrs` · **Output:** `Price_USD`

Section 1: Setup & Dataset

Q1. Import libraries and load `laptop_price_gbr.csv` into a DataFrame. Check its shape and data types.

```
In [1]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import GradientBoostingRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
```

```
pd.set_option("display.width", 120)
np.random.seed(42)
```

```
In [2]: # Reproducible synthetic dataset with the schema used in this project.
# (Swap this cell for `pd.read_csv("laptop_price_gbr.csv")` if you have the real file.)
n = 10000

ram_gb = np.random.choice([4, 8, 16, 32, 64], size=n, p=[0.10, 0.30, 0.35, 0.20, 0.05])
storage_gb = np.random.choice([128, 256, 512, 1024, 2048], size=n, p=[0.10, 0.25, 0.35, 0.20, 0.10])
screen_size_inch = np.round(np.random.uniform(11.0, 17.3, size=n), 1)
weight_kg = np.round(np.random.uniform(0.9, 3.2, size=n), 2)
battery_life_hrs = np.round(np.random.uniform(4.0, 20.0, size=n), 1)

# Non-linear price signal (diminishing returns + interactions) plus noise,
# so a non-linear model like Gradient Boosting has a real edge over Linear Regression.
price_usd = (
    250
    + 55 * np.sqrt(ram_gb) * 6
    + 0.28 * storage_gb
    + 40 * screen_size_inch
    - 120 * weight_kg
    + 15 * battery_life_hrs
    + 0.015 * (ram_gb * storage_gb) ** 0.5 * 40
    - 0.6 * (screen_size_inch - 14) ** 2
    + np.random.normal(0, 90, size=n)
)
price_usd = np.round(np.clip(price_usd, 300, None), 2)

df = pd.DataFrame({
    "RAM_GB": ram_gb,
    "Storage_GB": storage_gb,
    "ScreenSize_inch": screen_size_inch,
    "Weight_kg": weight_kg,
    "BatteryLife_hrs": battery_life_hrs,
    "Price_USD": price_usd,
})

print(df.shape)
df.info()
df.describe()
```

```
(10000, 6)
<class 'pandas.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 6 columns):
#   Column          Non-Null Count  Dtype
---  -
0   RAM_GB          10000 non-null  int64
1   Storage_GB      10000 non-null  int64
2   ScreenSize_inch 10000 non-null  float64
3   Weight_kg       10000 non-null  float64
4   BatteryLife_hrs 10000 non-null  float64
5   Price_USD       10000 non-null  float64
dtypes: float64(4), int64(2)
memory usage: 468.9 KB
```

Out[2]:

	RAM_GB	Storage_GB	ScreenSize_inch	Weight_kg	BatteryLife_hrs	Price_USD
count	10000.000000	10000.000000	10000.000000	10000.000000	10000.000000	10000.000000
mean	17.718800	673.395200	14.150510	2.047037	11.95012	2297.660210
std	13.788834	547.041777	1.806844	0.664717	4.62850	552.067665
min	4.000000	128.000000	11.000000	0.900000	4.00000	1082.670000
25%	8.000000	256.000000	12.600000	1.470000	7.90000	1892.587500
50%	16.000000	512.000000	14.200000	2.050000	11.90000	2215.475000
75%	16.000000	1024.000000	15.700000	2.620000	16.00000	2638.245000
max	64.000000	2048.000000	17.300000	3.200000	20.00000	4471.560000

Q2. Confirm the dataset has no missing values or duplicate rows.

```
In [3]: print(df.isnull().sum())
print("\nDuplicate Rows:", df.duplicated().sum())
```

```
RAM_GB          0
Storage_GB      0
ScreenSize_inch 0
Weight_kg       0
BatteryLife_hrs 0
Price_USD       0
dtype: int64
```

Duplicate Rows: 0

Section 2: Train / Test Split

Q3. Separate the independent variables `X` and the dependent variable `y` (`Price_USD`).

```
In [4]: X = df[["RAM_GB", "Storage_GB", "ScreenSize_inch", "Weight_kg", "BatteryLife_hrs"]]
y = df["Price_USD"]
```

Q4. Split the dataset into training and testing sets (80-20 split, `random_state=42`).

```
In [5]: X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

print("Train size:", X_train.shape[0], "| Test size:", X_test.shape[0])
```

Train size: 8000 | Test size: 2000

Section 3: Baseline — Linear Regression

A quick baseline to compare against the Gradient Boosting model.

```
In [6]: lin_model = LinearRegression()
lin_model.fit(X_train, y_train)

lin_pred = lin_model.predict(X_test)
lin_r2 = r2_score(y_test, lin_pred)
lin_rmse = np.sqrt(mean_squared_error(y_test, lin_pred))

print("Linear Regression -> R2:", round(lin_r2, 4), "| RMSE:", round(lin_rmse, 2))
```

Linear Regression -> R2: 0.9413 | RMSE: 134.99

Section 4: Gradient Boosting Regressor

Q5. Build a `GradientBoostingRegressor` model with `n_estimators=100`, `learning_rate=0.1`, and `max_depth=3`. Train it on the training data.

```
In [7]: gbr_model = GradientBoostingRegressor(
        n_estimators=100,
        learning_rate=0.1,
        max_depth=3,
        random_state=42
    )

gbr_model.fit(X_train, y_train)
```

```
Out[7]: ▼ GradientBoostingRegressor ⓘ ?
        ► Parameters
```

Q6. Predict `Price_USD` for the test set.

```
In [8]: y_pred = gbr_model.predict(X_test)
        y_pred[:10]
```

```
Out[8]: array([1865.03888177, 2822.24797253, 2145.55757535, 1744.68433775,
        2653.1692323 , 1755.0821481 , 2533.38080259, 2457.68479705,
        2397.09758157, 1489.09482421])
```

Q7. Evaluate the model using MAE, MSE, RMSE, and R^2 score.

```
In [9]: mae = mean_absolute_error(y_test, y_pred)
        mse = mean_squared_error(y_test, y_pred)
        rmse = np.sqrt(mse)
        r2 = r2_score(y_test, y_pred)

        print("MAE:", round(mae, 2))
        print("MSE:", round(mse, 2))
        print("RMSE:", round(rmse, 2))
        print("R2 Score:", round(r2, 4))
```

```
MAE: 74.3
MSE: 8551.72
RMSE: 92.48
R2 Score: 0.9725
```

Q8. Check the training vs. testing score to confirm the model isn't overfitting.

```
In [10]: train_score = gbr_model.score(X_train, y_train)
        test_score = gbr_model.score(X_test, y_test)

        print("Training Score:", round(train_score, 4))
        print("Testing Score:", round(test_score, 4))
```

```
Training Score: 0.9746
Testing Score: 0.9725
```

Q9. Extract and plot the feature importances from the trained model. Which feature contributes most to predicting `Price_USD`?

```
In [11]: importances = pd.DataFrame({
        "Feature": X.columns,
        "Importance": gbr_model.feature_importances_
    }).sort_values("Importance", ascending=False)

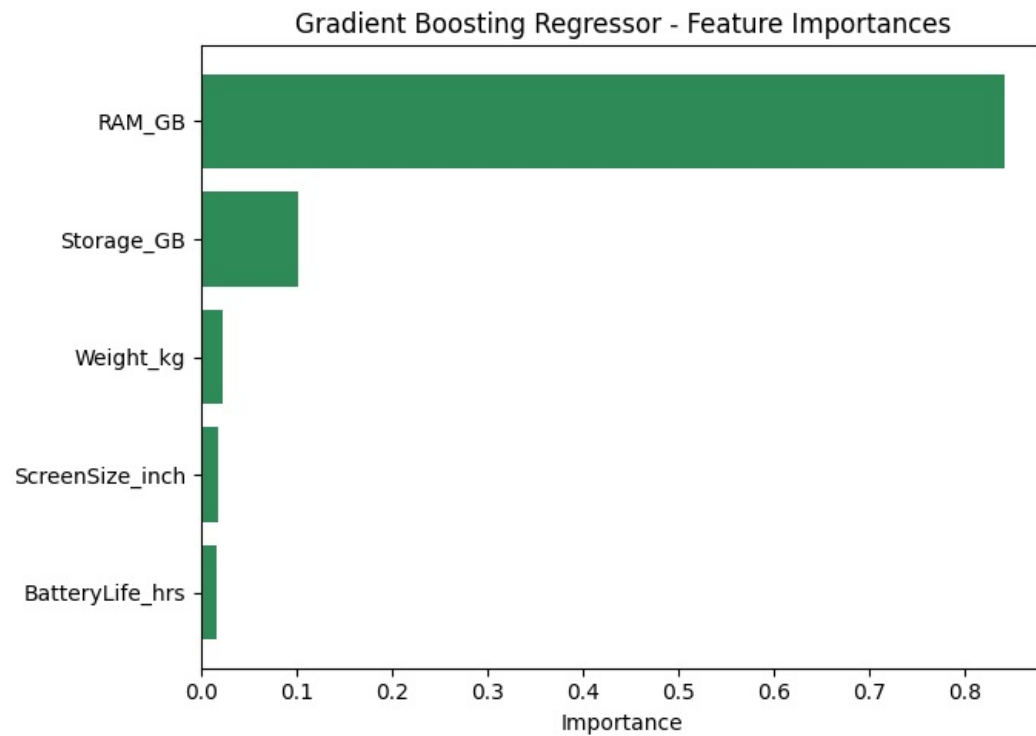
        print(importances)

        top_feature = importances.iloc[0]["Feature"]
        print("\nMost important feature:", top_feature)

        plt.figure(figsize=(7, 5))
        plt.barh(importances["Feature"], importances["Importance"], color="seagreen")
        plt.xlabel("Importance")
        plt.title("Gradient Boosting Regressor - Feature Importances")
        plt.gca().invert_yaxis()
        plt.tight_layout()
        plt.show()
```

	Feature	Importance
0	RAM_GB	0.840979
1	Storage_GB	0.102522
3	Weight_kg	0.022689
2	ScreenSize_inch	0.017287
4	BatteryLife_hrs	0.016523

Most important feature: RAM_GB



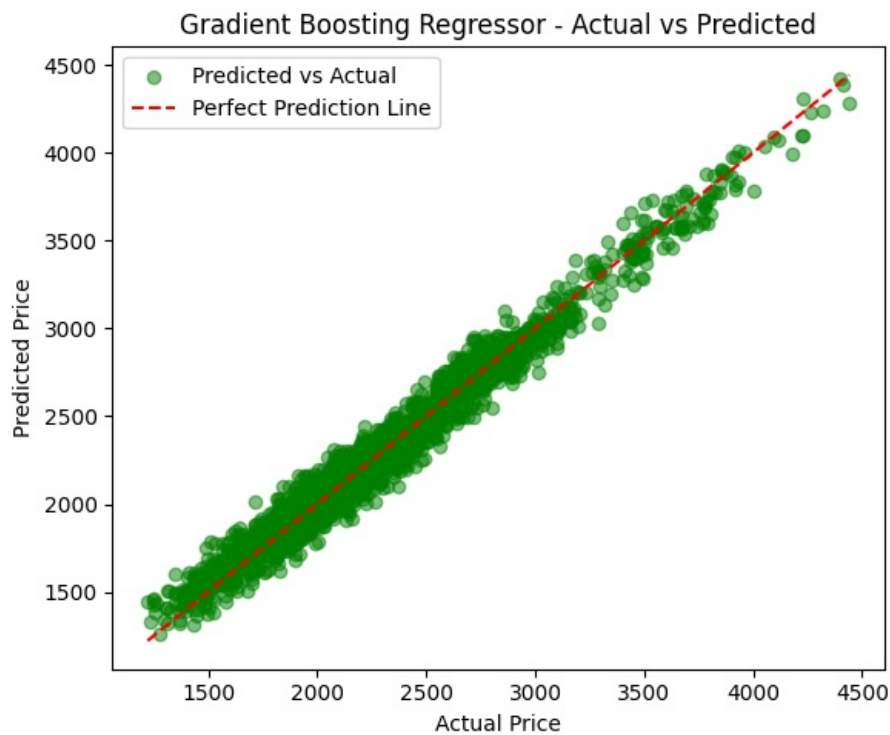
Q10. Create a scatter plot comparing Actual vs Predicted prices, with a diagonal reference line for perfect predictions.

```
In [12]: plt.figure(figsize=(6, 5))

plt.scatter(y_test, y_pred, color="green", alpha=0.5, label="Predicted vs Actual")

plt.plot(
    [y_test.min(), y_test.max()],
    [y_test.min(), y_test.max()],
    color="red",
    linestyle="--",
    label="Perfect Prediction Line"
)

plt.xlabel("Actual Price")
plt.ylabel("Predicted Price")
plt.title("Gradient Boosting Regressor - Actual vs Predicted")
plt.legend()
plt.tight_layout()
plt.show()
```



Section 5: Linear Regression vs. Gradient Boosting Regressor

Q11. Compare the R^2 score of Gradient Boosting with the Linear Regression baseline from Section 3 — which model performs better on this non-linear relationship, and why?

```
In [13]: comparison = pd.DataFrame({
    "Model": ["Linear Regression", "Gradient Boosting Regressor"],
    "R2 Score": [round(lin_r2, 4), round(r2, 4)],
    "RMSE": [round(lin_rmse, 2), round(rmse, 2)]
})

comparison
```

```
Out[13]:
```

	Model	R2 Score	RMSE
0	Linear Regression	0.9413	134.99
1	Gradient Boosting Regressor	0.9725	92.48

Takeaway: Gradient Boosting Regressor builds an ensemble of shallow decision trees in sequence, each one correcting the errors of the last. That lets it capture non-linear effects and feature interactions (like diminishing returns on RAM/storage, or the screen-size penalty) that a single linear equation cannot represent — which is why it typically beats plain Linear Regression on datasets with non-linear pricing behavior like this one.